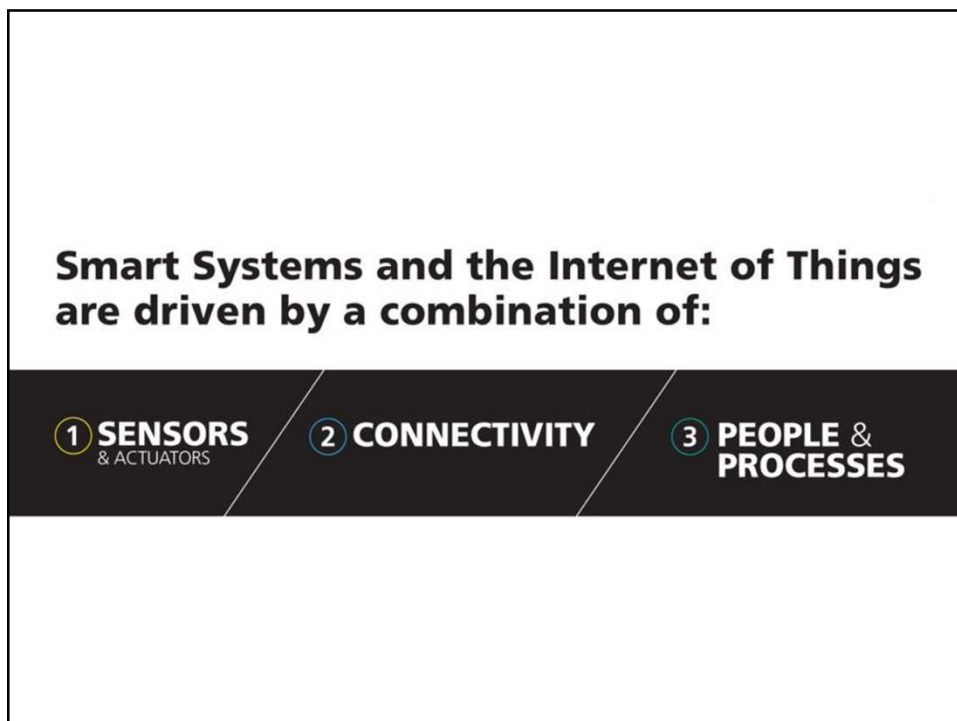
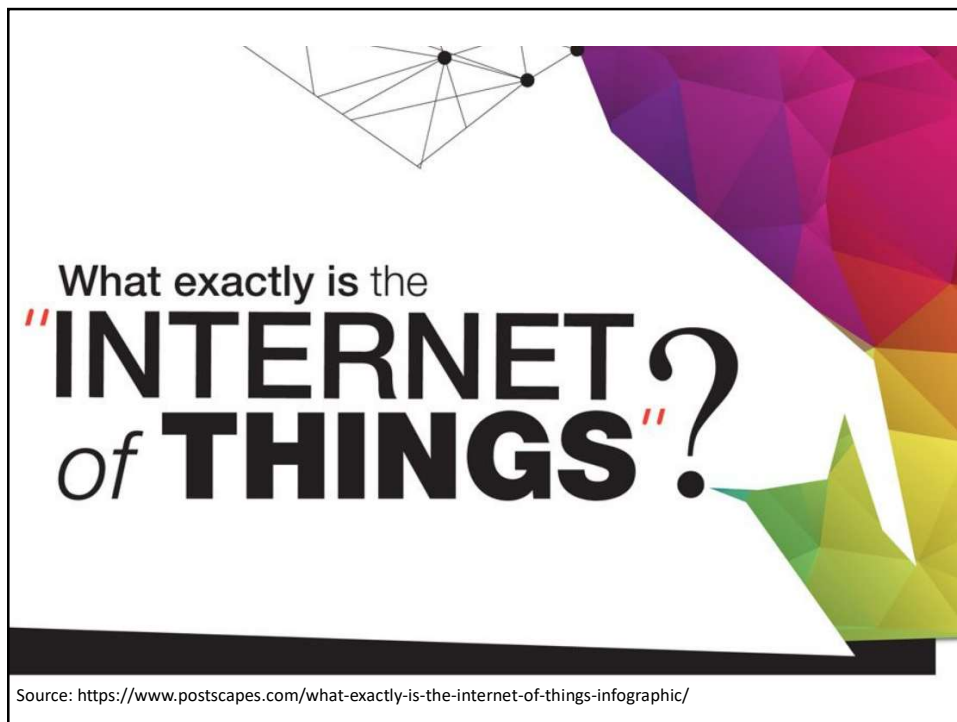


IoT Overview

UNIT 1

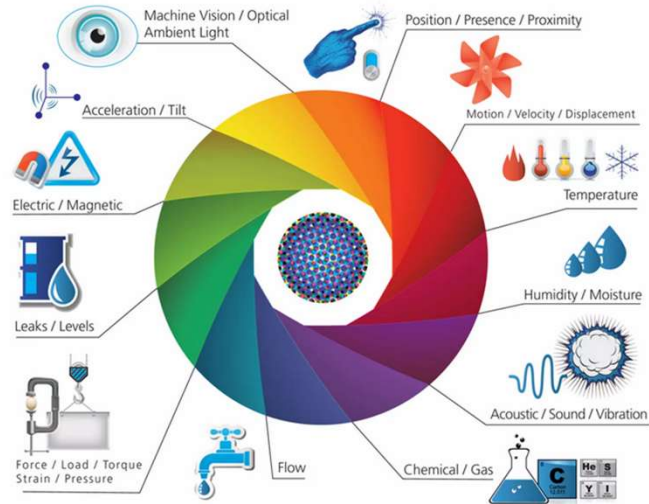
Contents

- Introduction,
- Physical design of IoT, (T1)
- Logical design of IoT, (T1)
- IoT architectural view, (T2)
- Sources of IoT, (T2)
- M2M communication, (T2)
- Examples of IoT, (T2)
- Communication Technologies, (T2)
- IoT levels and deployment templates (T1)



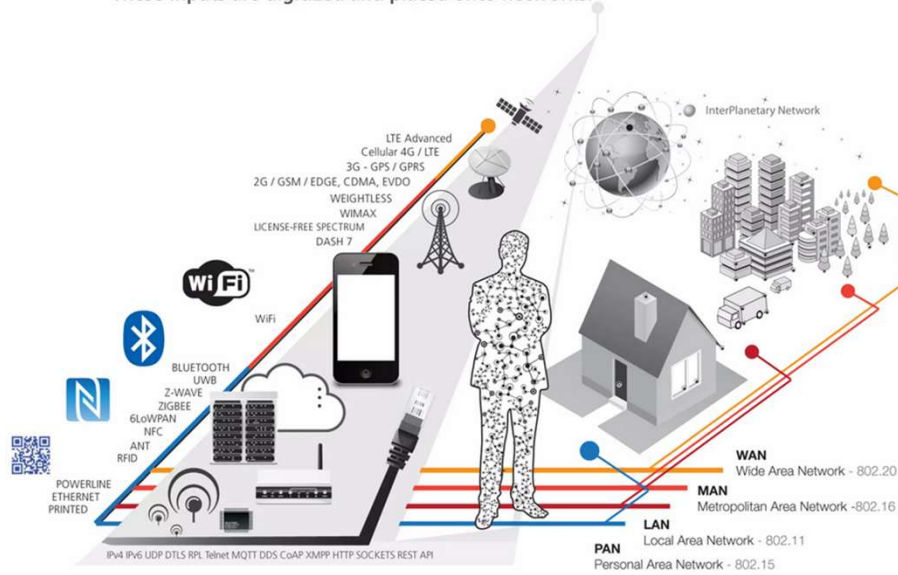
1 SENSORS & ACTUATORS

We are giving our world a digital nervous system. Location data using GPS sensors. Eyes and ears using cameras and microphones, along with sensory organs that can measure everything from temperature to pressure changes.



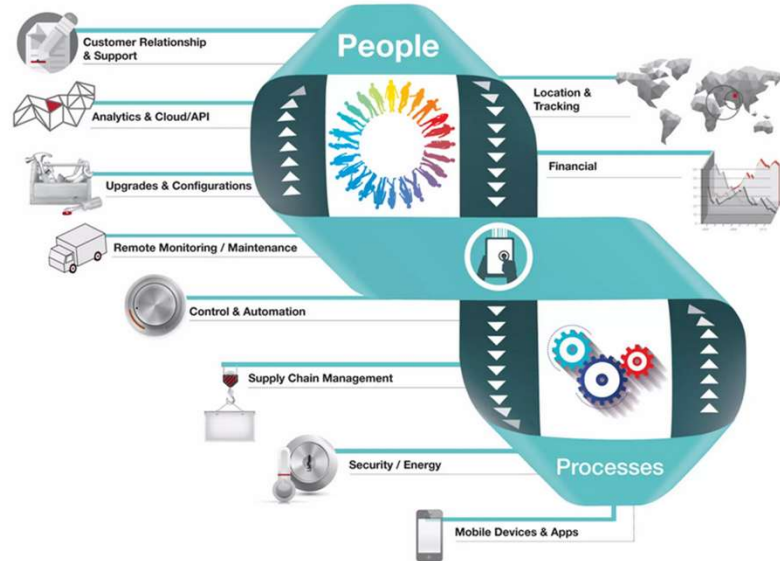
2 CONNECTIVITY

These inputs are digitized and placed onto networks.



3 PEOPLE & PROCESSES

These networked inputs can then be combined into bi-directional systems that integrate data, people, processes and systems for better decision making.



The interactions between these entities are creating new types of smart applications and services.

• SENSORS + CONNECTIVITY + PEOPLE + PROCESSES

Starting with popular connected devices already on the market



SMART THERMOSTATS

nest



Save resources and money on your heating bills by adapting to your usage patterns and turning the temperature down when you're away from home.

CONNECTED CARS

CAR2GO



Tracked and rented using a smartphone. Car2Go also handles billing, parking and insurance automatically.

ACTIVITY TRACKERS

BASIS



Continuously capture heart rate patterns, activity levels, calorie expenditure and skin temperature on your wrist 24/7.

SMART OUTLETS

belkin



Remotely turn any device or appliance on or off. Track a device's energy usage and receive personalized notifications from your smartphone.

PARKING SENSORS

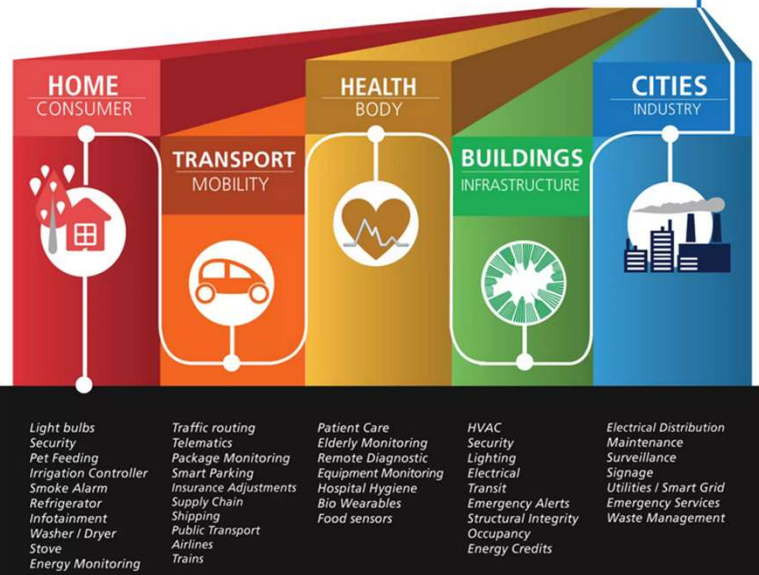
STREETLINE



Using embedded street sensors, users can identify real-time availability of parking spaces on their phone. City officials can manage and price their resources based on actual use.

And quickly advancing

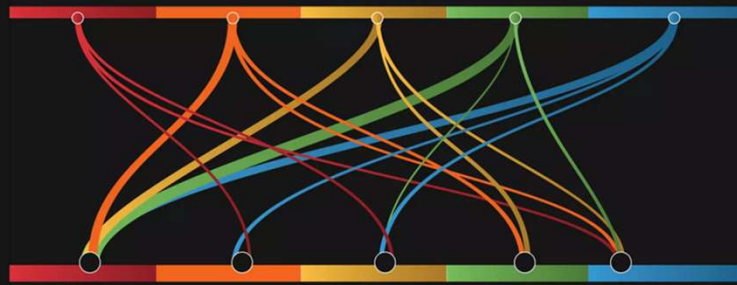
TO DIVERSE APPLICATIONS



Things get interesting when these connected devices and services start creating

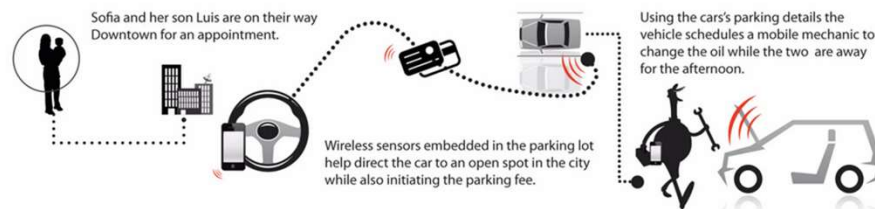
COMPOUND APPLICATIONS

within their own verticals and across industries:



FOR EXAMPLE

TRANSPORTATION + SMART CITIES



In Downtown San Francisco 20-30% of all traffic congestion is caused by people hunting for a parking spot.

- San Francisco Municipal Transportation Agency (SFMTA)

HEALTHCARE + SMART HOME



Aging uncle Earl is still living isolated at his home and you are concerned about his safety.



Wireless sensors throughout his house help measure healthy activity levels, sleeping patterns and medication schedules.



Alerts are automatically sent to health care services and authorized family members if any abnormal activity is detected.

40 million adults age 65 and over will be living alone in the U.S, Canada and Europe.

- U.S. Department of Health and Human Services: Administration for Community Living (ACL)

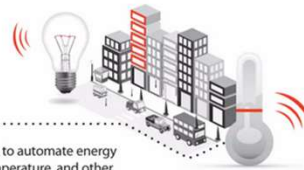
SMART BUILDINGS + MOBILITY



Anna is being pressured to reduce her company's expenses for their new corporate office.

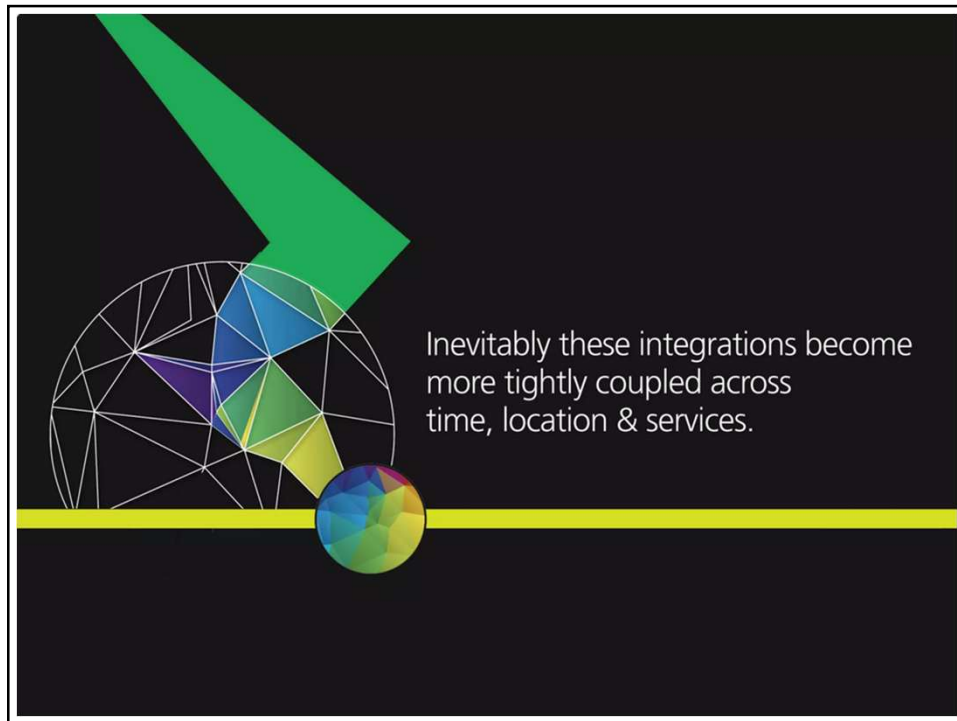


After speaking with experts she decides to install sensors to automate energy usage according to building occupancy, people flow, temperature, and other ambient conditions -- improving the building's overall efficiency.



Energy used by commercial and industrial buildings in the US creates nearly 50% of our national emissions of greenhouse gases.

- United States Environmental Protection Agency



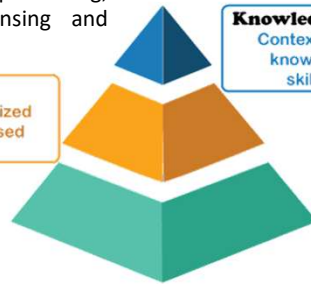
Introduction

Inferring information and knowledge from data

Inferred by filtering, processing, categorizing, condensing and contextualizing Data

Information

Contextualized categorized calculated and condensed data.



Inferred by organizing and structuring Information to achieve specific objectives

Knowledge

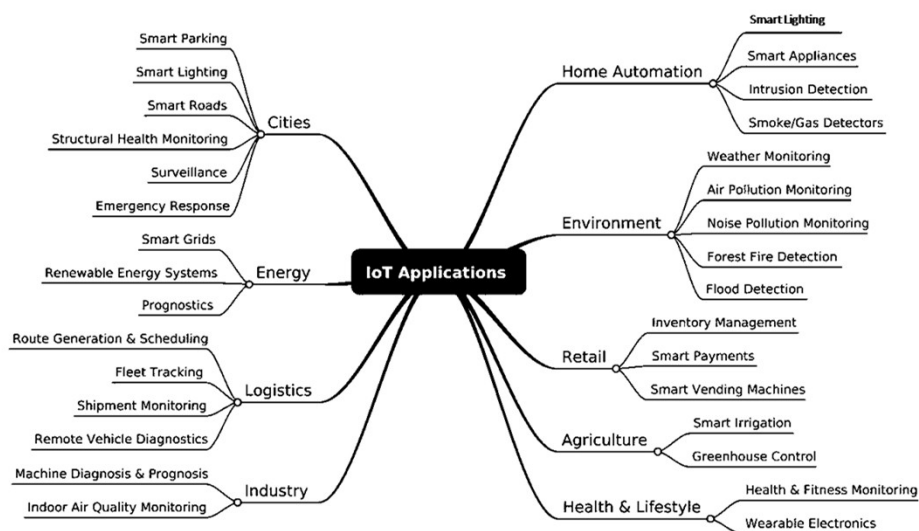
Contextualized information, know how, perceptions, skills experience.

Data

Facts and figures which are useful but in unorganized manner.

Raw and unprocessed data obtained from IoT devices or systems

Applications of IoT



Definition of IoT

A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual "things" have identities, physical attributes and virtual personalities, use intelligent interfaces, are seamlessly integrated into the information network, and often communicate data associated with users and their environments.

Characteristics of IoT

- Dynamic and self-adapting
- Self-configuring
- Interoperable Communication Protocols
- Unique identity
- Integrated into Information Network

Characteristics of IoT

- **Dynamic and self-adapting:**

- Dynamically adapt and take actions based on their operating conditions, user's context, or sensed environment.
- For example,
 - consider a surveillance system comprising of a number of surveillance cameras.
 - The surveillance cameras can adapt their modes (to normal or infra-red night modes)
 - lower resolution to higher resolution modes when motion is detected and alert nearby cameras to do the same.
 - In this example, the surveillance system is adapting itself based on the context and changing (e.g., dynamic) conditions.

- **Self-configuring:**

- IoT devices may have self-configuring capability, allowing a large number of devices to work together to provide certain functionality.
- These devices have the ability configure themselves setup the networking, and fetch latest software upgrades with minimal manual or user intervention.

Characteristics of IoT

- **Interoperable Communication Protocols:**

- IoT devices may support a number of interoperable communication protocols and can communicate with other devices and also with the infrastructure.

- **Unique identity:**

- Each IoT device has a unique identity and a unique identifier (such as an IP address or a URI).
- IoT systems may have intelligent interfaces which adapt based on the context, allow communicating with users and the environmental contexts.
- IoT device interfaces allow users to query the devices, monitor their status, and control them remotely, in association with the control, configuration and management infrastructure.

Characteristics of IoT

- **Integrated into Information Network:**

- IoT devices are usually integrated into the information network that allows them to communicate and exchange data with other devices and systems.
- IoT devices can be dynamically discovered in the network, by other devices and/or the network, and have the capability to describe themselves (and their characteristics) to other devices or user applications.
- **For example,**
 - a weather monitoring node can describe its monitoring capabilities to another connected node so that they can communicate and exchange data.
 - Integration into the information network helps in making IoT systems "smarter" due to the collective intelligence of the individual devices in collaboration with the infrastructure.
 - Thus, the data from a large number of connected weather monitoring IoT nodes can be aggregated and analyzed to predict the weather.

Physical Design of IoT

Generic Block Diagram of an IoT Device

IoT Protocols

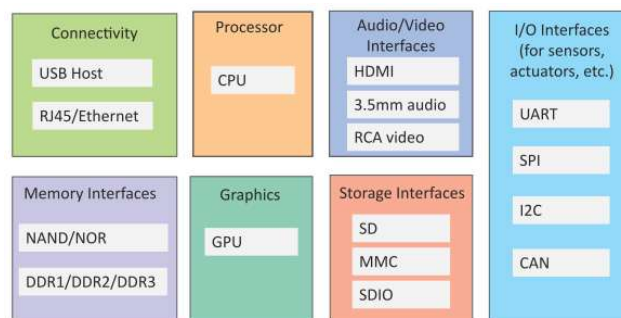
Physical Design of IoT

Physical Design of IoT

- The **"Things"** in IoT usually refers to IoT devices which have unique identities and can perform remote sensing, actuating and have monitoring capabilities.
- IoT devices can:
 - **Exchange data** with other connected devices and applications (directly or indirectly),
 - **Collect data** from other devices and process the data locally,
 - **Send the data** to centralized servers or cloud-based application back-ends for processing the data,
 - **Perform some tasks** locally and other tasks within the IoT infrastructure, based on temporal and space constraints

Physical Design of IoT

Generic Block Diagram of an IoT Device



- An IoT device may consist of several interfaces for connections to other devices, both wired and wireless.
 - I/O interfaces for sensors
 - Interfaces for internet connectivity
 - Memory and storage interfaces
 - Audio/video interfaces

Physical Design of IoT

IoT Protocols

Link Layer

- 802.3 – Ethernet
- 802.11 – WiFi
- 802.16 – WiMax
- 802.15.4 – LR-WPAN
- 2G/3G/4G/5G

Network/Internet Layer

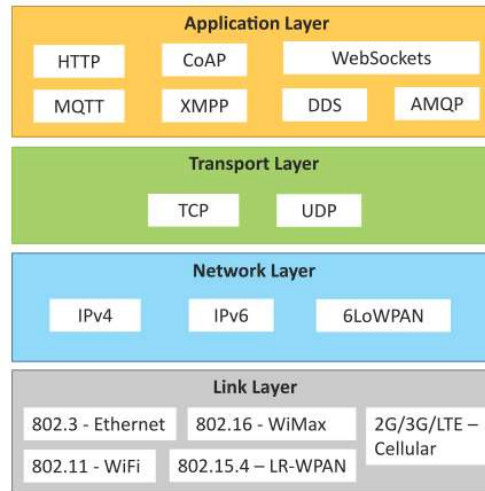
- IPv4
- IPv6
- 6LoWPAN

Transport Layer

- TCP
- UDP

Application Layer

- HTTP
- CoAP
- WebSocket
- MQTT
- XMPP
- DDS
- AMQP



Physical Design of IoT – IoT protocols

Link Layer

- Link Layer protocols determine how the data is physically sent over the networks physical layer or medium.
- The Scope of the Link Layer is the Local Network connection to which host is attached.
- Host on the same link exchange data packets over the link layer using the link layer protocol.
- Link layer determines how the packets are coded and signaled by the hardware device over the medium to which the host is attached.
- **802.3 Ethernet:**
 - collections of wired Ethernet standards
 - 802.3 10BASE5 Ethernet that uses coaxial cable as a shared medium,
 - 802.3.i is standard for 10BASE-T Ethernet over copper twisted pair connection,
 - 802.j → 10BASE-F , 802.ae → 10Gbps ethernet (Fiber)
 - 10 Mbps to 40 Gbps and the higher
 - a coaxial cable, twisted pair wire or and Optical fiber

Physical Design of IoT – IoT protocols

Link Layer

- **802.11 - WiFi:**
 - collection of wireless local area network (WLAN) communication standards,
 - For example,
 - 802.11a operates in the 5 GHz band,
 - 802.11b and 802.11g operate in the 2.4 GHz band,
 - 802.11n operates in the 2.4/5 GHz bands,
 - 802.11ac operates in the 5GHz band and
 - 802.11ad operates in the 60 GHz band.
 - These standards provide data rates from 1 Mb/s to up to 6.75 Gb/s.
- **802.16 - WiMax:**
 - IEEE 802.16 is a collection of wireless broadband standards,
 - provide data rates from 1.5 Mb/s to 1 Gb/s.
 - The recent update (802.16m) provides data rates of 100 Mbit/s for mobile stations and 1 Gbit/s for fixed stations.

Physical Design of IoT – IoT protocols

Link Layer

- **802.15.4 - LR-WPAN :**
 - collection of standards for low-rate wireless personal area networks (LR-WPANs).
 - form the basis of specifications for high level communication protocols such as ZigBee.
 - provide data rates from 40 Kb/s to 250 Kb/s.
 - low-cost and low-speed communication for power constrained devices.
- **2G/ 3G/ 4G - Mobile Communication :**
 - 2G including GSM and CDMA
 - 3G - including UMTS and CDMA2000
 - 4G - including LTE
 - IoT devices based on these standards can communicate over cellular networks, Data rates for these standards range from 9.6 Kb/s (for 2G) to up to 100 Mb/s (for 4G)

Network/Internet Layer

- responsible for sending of IP datagrams from the source to destination network.
- performs the host addressing and packet routing.
- the source and destination addresses are used to route datagrams from the source to destination across multiple networks.
- IP addressing schemes such as IPv4 or IPv6.
- **IPv4:**
 - 32-bit address scheme that allows total of 2^{32} or 4,294,967,296 addresses.
 - these addresses got exhausted in the year 2011.
 - IPv4 has been succeeded by IPv6.
 - IP protocols establish connections on packet networks, but do not guarantee delivery of packets.

Network/Internet Layer

- **IPv6:**
 - newest version of Internet protocol and successor to IPv4.
 - IPv6 uses 128-bit address scheme that allows total of 2^{128} or 3.4×10^{38} addresses.
- **6LoWPAN:**
 - 6LoWPAN (IPv6 over Low power Wireless Personal Area Networks) brings IP protocol to the low-power devices which have limited processing capability.
 - operates in the 2.4 GHz frequency range and provides data transfer rates of 250 Kb/s.
 - 6LoWPAN works with the 802.15.4 link layer protocol and defines compression mechanisms for IPv6 datagrams over IEEE 802.15.4-based networks

Transport Layer

- provide end-to-end message transfer capability independent of the underlying network.
- can be set up on connections, either using handshakes (as in TCP) or without handshakes/acknowledgements (as in UDP).
- provides functions such as error control, segmentation, flow control and congestion control.
- **Transmission Control Protocol (TCP)**
 - the most widely used transport layer protocol,
 - used by web browsers (along with HTTP, HTTPS application layer protocols), email programs (SMTP application layer protocol) and file transfer (FTP).
 - connection oriented and stateful protocol.
 - While IP protocol deals with sending packets, TCP ensures reliable transmission of packets in-order.
 - TCP also provides error detection capability
 - The flow control capability
 - The congestion control capability
 - congestion collapse which can lead to degradation of network performance

- **User Datagram Protocol (UDP):**
 - UDP is a connectionless protocol.
 - UDP is useful for time-sensitive applications with very small data units to exchange and do not want the overhead of connection setup.
 - is a transaction oriented and stateless protocol.
 - does not provide guaranteed delivery, ordering of messages and duplicate elimination.
 - Higher levels of protocol scan ensure reliable delivery or ensuring connections created are reliable.

Physical Design of IoT – IoT protocols

Application Layer

- Application layer protocols define how the applications interface with the lower layer protocols to send the data over the network.
- Application data is encoded by the application layer protocol and encapsulated in the transport layer protocol
- Port numbers are used for application addressing (for example port 80 for HTTP, port 22 for SSH, etc.).
- Protocols enable process-to-process connections using ports.
- **Hypertext Transfer Protocol (HTTP):**
 - Foundation of the World Wide Web (WWW).
 - Uses commands such as GET, PUT, POST, DELETE, HEAD, TRACE, OPTIONS, etc.
 - **request-response model** where a client sends requests to a server using the HTTP commands.
 - HTTP is a **stateless protocol** and each HTTP request is independent of the other requests.
 - HTTP client can be a browser or an application running on the client
 - HTTP protocol uses **Universal Resource Identifiers (URIs)** to identify HTTP resources.

Physical Design of IoT – IoT protocols

Application Layer

- **Hypertext Transfer Protocol (HTTP):**
 - Foundation of the World Wide Web (WWW).
 - Commands → GET, PUT, POST, DELETE, HEAD, TRACE, OPTIONS, etc.
 - **request-response model** where a client sends requests to a server using the HTTP commands.
 - is a **stateless protocol** and each HTTP request is independent of the other requests.
 - client can be a browser or an application running on the client
 - uses **Universal Resource Identifiers (URIs)** to identify HTTP resources.



Application Layer

- **Constrained Application Protocol (CoAP):**

- application layer protocol for machine-to-machine (M2M) applications, for constrained environments with constrained devices and constrained networks.
- A web transfer protocol and uses a **request-response model**, however it runs on top of **UDP** instead of TCP.
- uses a **client-server architecture** where clients communicate with servers using connectionless datagrams.
- designed to easily interface with HTTP.
- supports methods such as GET, PUT, POST, and DELETE.

- **WebSocket:**

- protocol allows **full-duplex** communication over a single socket connection for sending messages between client and server.
- Based on **TCP** and allows streams of messages to be sent back and forth between the client and server while keeping the TCP connection open.
- The client can be a browser, a mobile application or an IoT device.

Application Layer

- **Message Queue Telemetry Transport (MQTT):**

- light-weight messaging protocol based on the **publish-subscribe** model.
- **client-server** architecture where the client (such as an IoT device) connects to the server (also called MQTT Broker) and publishes messages to topics on the server.
- The broker forwards the messages to the clients subscribed to topics.
- well suited for devices that have **limited** processing and memory resources and the network bandwidth is low.

- **Extensible Messaging and Presence Protocol (XMPP):**

- protocol for **real-time communication** and **streaming XML** data between network entities.
- XMPP powers wide range of applications including messaging, presence, data syndication, gaming, multi-party chat and voice/video calls.
- XMPP is a decentralized protocol and uses a **client-server** architecture.
- XMPP supports both client-to-server and server-to-server communication paths.

Application Layer

- **Data Distribution Service (DDS):**
 - is a data-centric middleware standard for device-to-device or machine-to-machine communication.
 - **publish-subscribe** model where publishers (e.g. devices that generate data) create topics to which subscribers (e.g., devices that want to consume data) can subscribe.
 - Publisher → data distribution and the subscriber → for receiving published data.
 - quality-of-service (QoS) control and configurable reliability.
- **Advanced Message Queuing Protocol (AMQP):**
 - is an open application layer protocol for business messaging.
 - supports both point-to-point and publisher/subscriber models, routing and queuing.
 - brokers receive messages from publishers (e.g., devices or applications that generate data) and route them over connections to consumers (applications that process data).
 - Publishers publish the messages to exchanges which then distribute message copies to queues.
 - Messages are either delivered by the broker to the consumers which have subscribed to the queues or the consumers can pull the messages from the queues.

Logical Design of IoT

IoT functional Blocks

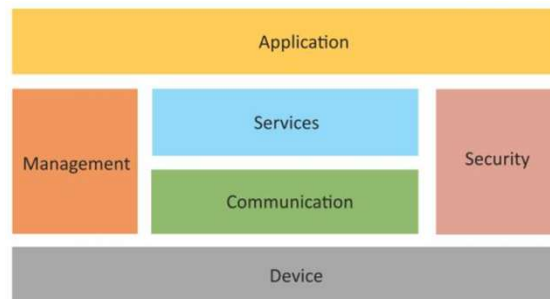
IoT Communication Models

IoT Communication APIs

Logical Design of IoT

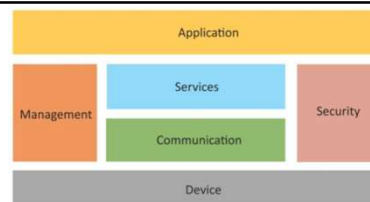
Logical Design of IoT

- Logical design of an IoT system refers to an abstract representation of the entities and processes without going into the low-level specifics of the implementation.
- An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication, and management.



Logical Design of IoT – IoT Functional Block

Logical Design of IoT

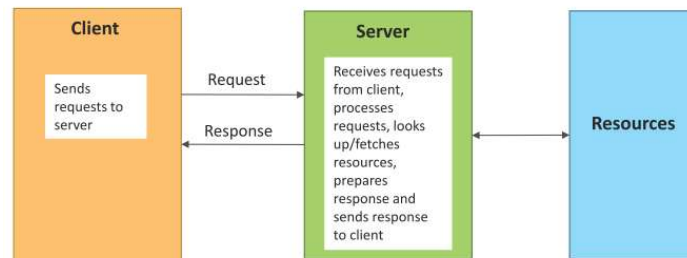


- **Device** : An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions.
- **Communication** : The communication block handles the communication for the IoT system.
- **Services** : An IoT system uses various types of IoT services such as services for device monitoring, device control services, data publishing services and services for device discovery.
- **Management** : Management functional block provides various functions to govern the IoT system..
- **Security**: Security functional block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- **Application** : IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view or analyze the processed data.

Logical Design of IoT – IoT Communication Models

Request-Response communication model

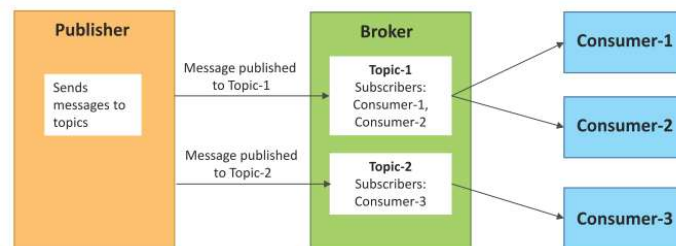
- Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests.
- When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client.



Logical Design of IoT – IoT Communication Models

Publish-Subscribe communication model

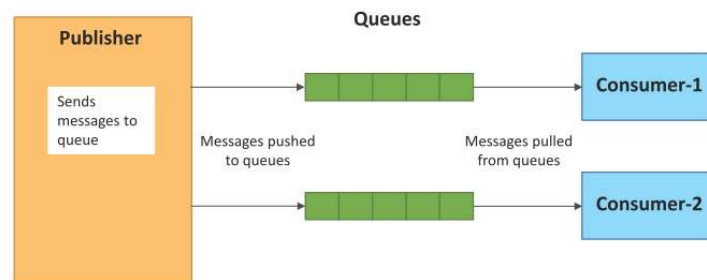
- Publish-Subscribe is a communication model that involves publishers, brokers and consumers.
- Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers.
- Consumers subscribe to the topics which are managed by the broker.
- When the broker receives data for a topic from the publisher, it sends the data to all the subscribed consumers.



Logical Design of IoT – IoT Communication Models

Push–Pull Communication Model

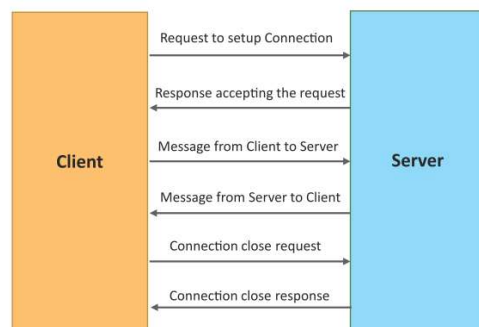
- Push–Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers.
- Queues help in decoupling the messaging between the producers and consumers.
- Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull data.



Logical Design of IoT – IoT Communication Models

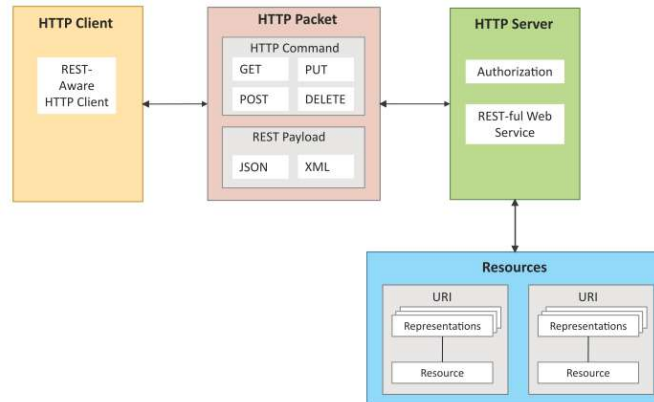
Exclusive Pair Communication Model

- Exclusive Pair is a bidirectional, fully duplex communication model that uses a persistent connection between the client and the server.
- Once the connection is set up it, remains open until the client sends a request to close the connection.
- Client and server can send messages to each other after connection setup.



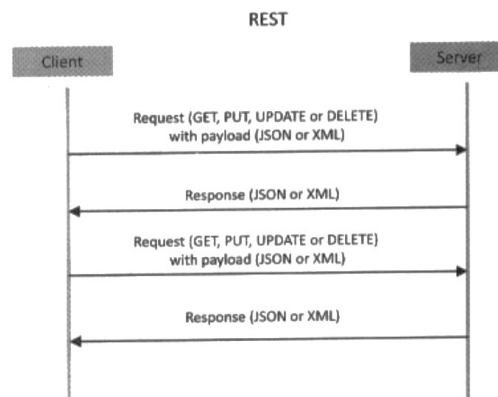
REST-based Communication APIs

- Representational State Transfer (REST) is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred.
- REST APIs follow the request–response communication model.
- REST architectural constraints apply to the components, connectors and data elements within a distributed hypermedia system.



REST-based Communication APIs

- REST architectural constraints are:
 - Client-Server
 - Stateless
 - Cache-able
 - Layered System
 - Uniform interface
 - Code on demand



REST-based Communication APIs

HTTP Method	Resource Type	Action	Example
GET	Collection URI	List all the resources in a collection	http://example.com/api/tasks/ (list all tasks)
GET	Element URI	Get information about a resource	http://example.com/api/tasks/1/ (get information on task-1)
POST	Collection URI	Create a new resource	http://example.com/api/tasks/ (create a new task from data provided in the request)
POST	Element URI	Generally not used	
PUT	Collection URI	Replace the entire collection with another collection	http://example.com/api/tasks/ (replace entire collection with data provided in the request)
PUT	Element URI	Update a resource	http://example.com/api/tasks/1/ (update task-1 with data provided in the request)
DELETE	Collection URI	Delete the entire collection	http://example.com/api/tasks/ (delete all tasks)
DELETE	Element URI	Delete a resource	http://example.com/api/tasks/1/ (delete task-1)

WebSocket-based Communication APIs

- WebSocket APIs allow bi-directional, full duplex communication between clients and servers.
- WebSocket APIs follow the exclusive pair communication model.
- WebSocket APIs reduce the network latency as there is no overhead for connection setup and termination requests for each message.

WebSocket Protocol

